

5장 같은 민원이 두 번 들어와도 데이터가 틀어지지 않게 하려면

4주차 · Kafka

4장은 컨슈머(consumer)가 처리 후 커밋(commit) 전에 죽으면 같은 메시지가 다시 수신된다는 것을 실측으로 남겼다. 유실은 되돌릴 수 없지만 중복은 교정할 수 있으므로, 실무는 중복을 감수하고 하류에서 걸러내는 쪽을 기본값으로 삼는다. 이 장은 그 "하류"를 실제로 만든다. 무엇을 같은 이벤트로 볼지 정하고, 그 판정 기준을 저장소에 걸고, 재처리를 몇 번 돌려도 결과가 그대로임을 지문 해시로 확인한다.

1. 오늘의 질문

- 같은 민원이 두 번 들어오는 것은 버그일까, 정상 동작의 부작용일까?
- 납품 문서에 적힌 "정확히 한 번(exactly-once) 처리"는 무엇을 보장한다는 뜻일까?
- 같은 시민이 어제와 오늘 같은 내용을 접수하면 중복일까, 별건일까?
- 재처리를 몇 번 돌려도 데이터가 그대로라는 것을 어떻게 증명할까?

2. 행사장 입장 명부로 보는 이 장

참가자에게 고유 번호가 적힌 손목띠를 채우고 입장할 때마다 명부에 적는 행사장을 생각하자. 이 장의 네 가지 결정이 명부 운영의 네 가지 결정과 하나씩 대응한다.

행사장 입장 명부	이 장의 개념
손목띠에 적힌 고유 번호	중복 제거 키 — 무엇이 같은 참가자인가
번호를 등록 창구에서 한 번만 발급하는가, 입장할 때마다 새로 주는가	키 발급 시점 — 재시도 경로에서 불변인가
명부 자체에 "같은 번호는 두 줄이 될 수 없다"를 새겨 두는가	유니크 제약 — 방어선을 코드가 아니라 스키마에
두 번째 입장을 지울까, 방문 횟수 칸을 올릴까	충돌 처리 — 무시할까 병합할까

가장 중요한 대응은 두 번째 줄이다. 손목띠 번호를 입장할 때마다 새로 발급하면 같은 사람이 명부에 여러 줄로 남는다. 번호가 있느냐보다 그 번호가 언제 발급되어 얼마나 오래 변하지 않느냐가 중복 제거의 성패를 가른다.

비유가 깨지는 지점이 하나 있다. 행사장은 접수원이 눈으로 확인하고 손으로 적으므로 확인과 기재 사이에 다른 사람이 끼어들지 않지만, 실제 시스템은 창구가 여러 개인 데다 각 창구가 서로를 보지 못한다. 그래서 5절에서 보듯 "확인하고 적는다"는 규칙을 사람에게 맡기면 뚫린다.

3. 중복은 버그가 아니라 재시도의 그림자

중복은 대개 코드가 틀려서 생기지 않는다. 유실을 막으려고 다시 보내거나 다시 처리한 결과다. 안전장치가 있는 자리마다 중복의 가능성이 함께 생긴다고 보는 편이 정확하다.

Producer가 이벤트를 보내면 브로커는 로그에 기록하고 확인 응답(ack)을 보낸다. 이 응답이 네트워크에서 사라지면 Producer는 타임아웃 뒤 전송이 실패했다고 판단해 같은 이벤트를 다시 보내고, 브로커에는 같은 이벤트가 두 번 기록된다. 이때 Producer가 아는 것은 "응답이 오지 않았다"뿐이며, 기록이 안 된 것인지 응답만 사라진 것인지 구분할 방법이 없다. 이 구분 불가능성이 분산 시스템 중복의 뿌리다. 재전송을 포기하면 유실을 감수해야 하고, 재전송하면 중복을 감수해야 한다.

중복이 생기는 자리는 네 곳이다.

- **Producer** — 확인 응답이 유실되어 재전송한다. 브로커에 같은 이벤트가 두 건 남는다.
- **브로커** — 스스로 중복을 만들지는 않는다. 다만 리더 교체가 Producer 재시도와 겹치면 중복이 남을 수 있다.
- **Consumer** — 4장이 실측한 경로다. 처리 후 커밋 전에 크래시(crash)하면 재시작한 컨슈머가 옛 커밋 위치부터 다시 읽는다.
- **DB** — "저장 성공, 커밋 실패" 뒤에 재시작하면 같은 이벤트를 다시 저장한다. 배치 재실행과 리플레이(replay) 재처리도 결과가 같다.

앞의 세 단계에는 각각 대응책이 있지만, 어느 것도 "최종 데이터에 중복이 없다"를 보장하지 못한다. Producer의 재시도 억제는 앱이 재시작한 뒤의 재전송을 막지 못하고, 커밋 시점 설계는 at-least-once를 고른 이상 정의상 중복을 남긴다. **저장소에 방어선이 없으면 상류의 모든 중복이 그대로 데이터가 된다.** 그래서 이 장의 무게중심은 마지막 단계인 DB에 있다.

전달 의미론 세 가지

4장에서 커밋 시점을 옮겨 가며 경험한 것을 정의로 정리하면 세 가지다. **at-most-once**는 최대 한 번 전달하며 중복은 없지만 유실이 가능하다. 4장 시나리오 B가 남긴 유실 20건이 이 경우다. **at-least-once**는 최소 한 번 전달하므로 유실은 없지만 중복이 가능하고, 4장 시나리오 C의 중복 10건이 여기에 해당한다. 4장에서 커밋을 처리보다 앞세울지 뒤로 미룰지 고른 것이 사실은 이 둘 중 하나를 고른 것이었다.

세 번째인 **exactly-once**를 읽을 때 주의할 것이 있다. 네트워크로 메시지를 주고받는 한 "정확히 한 번 전달"을 절대적으로 보장하기는 원리적으로 어렵다. 발신자는 수신 여부를 확인 응답으로만 알 수 있는데 그 응답이 또 유실될 수 있기 때문이다(두 장군 문제). 그래서 실무의 exactly-once는 대개 "여러 번

전달되어도 **정확히 한 번 반영**된 것과 같은 결과"를 뜻한다. 납품 문서에서 "exactly-once 지원"이라는 문구를 보면 그것이 전달 보장인지 반영 보장인지 되묻는 것이 검수의 출발점이다.

그 반영을 만드는 성질이 **멱등(idempotent)**이다. 같은 연산을 여러 번 적용해도 결과가 한 번 적용한 것과 같으면 멱등이다. 이미 눌린 엘리베이터 버튼을 다섯 번 더 눌러도 호출이 하나로 유지되는 것과 구조가 같다. 반대의 예가 "잔액 += 입금액" 같은 누적 연산으로, 같은 이벤트가 두 번 반영되면 값이 틀어진다. 이 장의 목표를 한 문장으로 줄이면 **모든 쓰기를 멱등 연산으로 만드는 것이다**.

중복 한 건을 실제로 들여다보면

4장 시나리오 C가 남긴 처리 기록에서 중복 이벤트 alo-000003의 두 기록을 꺼내 보자. 1회차(크래시 전)는 파티션(partition) 0, 오프셋(offset) 0, 처리 시각 01:59:20이고, 2회차(재시작 후)는 파티션 0, 오프셋 0, 처리 시각 01:59:35이다. (파티션, 오프셋)이 완전히 같다는 것이 핵심이다. 브로커의 로그에 이 이벤트는 한 건뿐이고, 컨슈머가 같은 위치를 두 번 읽었다는 뜻이다.

그러므로 이 중복은 Producer 재전송이 아니라 Consumer 재수신이며, 15초의 시차는 4장에서 본 세션 타임아웃(session timeout) 10초와 재할당 대기의 흔적이다. 중복을 발견했을 때 (파티션, 오프셋)이 같은지만 보면 어느 단계에서 생긴 중복인지 바로 판별할 수 있다.

4. "같은 이벤트"를 정하는 것은 기술이 아니라 업무다

저장소가 중복을 걸러내려면 "이 두 행은 같은 이벤트다"를 판정할 기준이 있어야 한다. 그 기준이 **중복 제거 키(deduplication key)**다. 그런데 같은 민원인이 같은 내용을 이틀 연속 접수한 경우가 중복인지 별건인지는 시스템이 아니라 업무 규정이 정한다. 키 선택이 기술 결정이 아니라 업무 정의인 이유다.

키로 쓸 수 있는 후보는 성격이 다르다.

- **생성 시점의 고유 ID** — 이벤트를 만드는 쪽이 붙이는 ID다. 재전송·재수신을 확실히 걸러내지만, 재접수처럼 사람이 두 번 한 행위는 서로 다른 ID를 받으므로 묵지 못한다.
- **업무 식별자(민원 접수번호)** — 업무적 의미가 분명해 재접수 판정에도 쓸 수 있지만, 접수번호가 발급되기 전 단계의 이벤트에는 아예 없다.
- **내용 해시** — 원천이 ID를 주지 않을 때의 차선택책이다. 필드 하나만 달라져도 다른 키가 되고, 반대로 같은 위치의 불법주차 민원 두 건처럼 별건을 중복으로 오판할 위험이 있다.

어느 후보도 기술적 중복 제거와 업무적 중복 판정을 동시에 해내지 못한다. 그래서 실무 정석은 층위를 나누는 것이다. **재시도·재수신 같은 기술적 중복은 생성 시점 고유 ID로 거르고, 재접수 같은 업무적 중복은 별도 규칙으로 다룬다**. 두 층위를 키 하나로 해결하려는 시도가 흔한 실패 원인이다.

언제 발급하느냐가 전부를 가른다

같은 UUID를 써도 발급하는 위치에 따라 결과가 정반대가 된다. 이벤트를 만들 때 ID를 붙여 두면 재전송에서도 같은 ID가 실려 나가므로 저장소가 두 번째 도착을 충돌로 잡아낸다. 반대로 전송 함수 안에서 ID를 발급하면 재전송할 때마다 새 ID가 붙어, 저장소 눈에는 서로 다른 이벤트가 되어 중복이 그대로 통과한다.

뒤쪽 구현도 "모든 이벤트에 고유 ID가 있다"는 요구는 완전히 만족한다. 검수에서 "고유 ID가 있는가"만 묻고 "그 ID가 재시도 경로에서 불변인가"를 묻지 않으면 이 결함을 그대로 통과시킨다. 4장 Producer가 이벤트를 만들 때 ID를 붙이고 전송 루프에서는 건드리지 않은 것이 앞쪽 구현이며, alo-000003이 크래시와 재시작을 관통해 불변이었기에 저장소가 두 기록을 같은 이벤트로 판정할 수 있었다.

이 원칙은 이 교재가 만든 것이 아니다. 결제 API는 같은 구조를 **멱등성 키(idempotency key)** 라는 이름으로 쓴다. 클라이언트가 요청마다 고유 값을 만들어 보내고, 네트워크 오류로 실패하면 같은 키로 재시도하며, 서버는 이미 본 키면 앞선 결과를 그대로 돌려준다. 키를 생성 시점에 발급하고, 재시도에 같은 키를 쓰고, 최종 판정은 서버가 내린다는 세 가지가 이 장의 설계와 그대로 겹친다.

재접수는 통과하는 것이 정상이다

같은 시민이 같은 불법주차를 오늘 접수하고, 처리가 안 됐다며 내일 다시 접수했다고 하자. 두 접수는 생성 시점에 서로 다른 event_id를 받으므로 유니크 제약을 통과해 두 행으로 남는다. 이것은 결함이 아니라 올바른 동작이다. 접수 행위가 실제로 두 번 있었고, 감사 관점에서는 둘 다 기록되어야 한다.

"사실상 같은 민원이니 묶자"는 판단은 그 위의 업무 규칙 계층에서 다룬다. 같은 위치·같은 유형·N일 이내라면 병합 검토 대상으로 **표시**하고 사람이 확인하게 하지, 자동으로 지우지 않는다. 기술 키가 지우는 것은 "같은 이벤트의 반복 전달"이지 "비슷한 사건의 반복 발생"이 아니다. 이 구분이 무너지면 시민의 정당한 재민원이 소리 없이 사라진다.

5. 방어선은 코드가 아니라 스키마에 둔다

키를 정했으면 그 키로 중복을 막을 자리를 정해야 한다. 가장 먼저 떠오르는 방법은 애플리케이션 코드에서 "저장 전에 있는지 확인하고 없으면 저장"이다. 컨슈머 두 대가 같은 이벤트를 거의 동시에 처리하면 이 방법이 어떻게 뚫리는지 보자.

컨슈머 1이 조회해서 이벤트 X가 없음을 확인한다. 그 직후 컨슈머 2도 조회하는데, 아직 아무도 넣지 않았으므로 역시 없다고 나온다. 이어서 컨슈머 1이 X를 넣어 성공하고, 컨슈머 2도 X를 넣어 성공한다. 두 컨슈머 모두 규칙을 성실히 지켰는데 행이 두 개 생겼다. 확인과 저장 사이의 틈에 상대가 끼어드는 **경쟁 조건(race condition)** 때문이다.

키에 **유니크 제약(unique constraint)** 이 걸려 있으면 컨슈머 2의 삽입은 저장소 안에서 원자적으로 거부된다. 방어선을 코드가 아니라 스키마에 두는 이유가 여기 있다. 코드는 인스턴스 수만큼 복제되어 서로를 모르지만, 제약은 데이터가 놓이는 단 한 곳에서 지켜진다. 검수자가 "중복 방어가 어디 있는가"를

물을 때 가장 강한 답이 애플리케이션 코드가 아니라 DDL의 `UNIQUE` 한 줄인 것도 같은 이유다.

충돌을 어떻게 처리할 것인가

제약에 걸린 삽입, 즉 충돌을 처리하는 방식은 세 가지다.

- **실패** — 오류를 내고 적재를 멈춘다. 중복 도착 자체가 이상 신호인 테이블에서 쓴다.
- **무시(INSERT OR IGNORE)** — 충돌한 행을 조용히 건너뛴다. 재전송이 항상 같은 내용일 때 적합하지만, 중복이 있었다는 사실 자체가 남지 않는다.
- **병합(UPSERT)** — 기존 행을 갱신한다. 재전송이 갱신을 담고 있거나, 중복이 도착했다는 사실을 남기고 싶을 때 쓴다.

선택 기준은 한 질문으로 정리된다. "**두 번째 도착이 새 정보를 담고 있는가?**" 담고 있지 않고 중복 도착이 이상 신호라면 실패, 정상이면서 상태 갱신을 담을 수 있으면 병합, 항상 같은 내용이고 중복 횟수를 지표로 쓸 생각이 없으면 무시다. 같은 시스템 안에서도 테이블의 역할에 따라 답이 달라진다. 민원의 현재 상태를 담는 테이블에 무시를 쓰면 최초 상태에 고정되어 이후 갱신이 모두 버려진다.

UPSERT 한 줄 읽기

```
INSERT INTO complaints_upsert VALUES (?, ?, ?, ?, ?, 1)
ON CONFLICT(event_id) DO UPDATE SET
    last_seen_at = excluded.last_seen_at, seen_count = seen_count + 1
```

첫 줄은 처음 보는 이벤트면 행을 만들고 관찰 횟수를 1로 시작하라는 뜻이다. `ON CONFLICT(event_id)` 는 `event_id` 유니크 제약에 걸렸을 때, 즉 이미 본 이벤트일 때를 가리키고, `DO UPDATE SET` 이 그때의 행동이다. `excluded` 는 충돌을 일으킨 새 행, 즉 들어오려다 거부된 값을 가리키는 예약 이름이다. `first_seen_at` 은 갱신 목록에 없으므로 최초 삽입값이 그대로 보존된다. 무엇을 갱신하고 무엇을 보존할지가 SQL 한 줄에 드러나는 것이 UPSERT의 장점이다.

이 설계는 중복을 버리지 않고 관찰 데이터로 바꾼다. `seen_count > 1` 인 행의 수를 세면 상류 파이프라인이 얼마나 자주 재시도·재처리를 했는지가 저장소 쪽에서 역으로 보인다. 다만 이 값을 "장애로 인한 재수신"으로 바로 읽으려면 장애 재수신과 계획된 정기 리플레이를 구분할 원인 필드가 함께 있어야 한다. 둘을 섞어 세면 계획된 재처리까지 장애로 잡혀 지표가 부풀어 오른다.

이 세 방식은 실습의 SQLite에만 있는 기능이 아니다. PostgreSQL도 `INSERT ... ON CONFLICT (event_id) DO UPDATE SET ...` 으로 골격이 같고, SQLite의 `excluded` 에 해당하는 것이 `EXCLUDED` 다. 다만 `ON CONFLICT` 는 SQL 표준이 아니라 두 제품이 공유하는 확장 문법이고, ISO 표준에 가까운 것은 Oracle-SQL Server가 지원하는 `MERGE` 문이다. 문법 차이만 흡수하면 실습에서 만든 방어선이 운영 DB로 거의 그대로 옮겨진다.

같은 방어선이 집계 저장에도 필요하다. 늦게 도착한 민원을 반영해 "23시대 12건"을 다시 계산하면 새 결과는 "23시대 13건"이다. 아무 제약 없이 삽입하면 23시대 행이 12와 13 두 개가 되어 어느 쪽이 맞는지 알 수 없다. 집계 구간을 키로 한 UPSERT면 12가 13으로 덮어써져 항상 최신 재계산만 남는다. 이 지연

이벤트를 언제까지 기다릴지 정하는 문제는 6장에서 스트리밍 엔진으로 직접 다룬다.

6. 실습 5.1 중복 이벤트 주입과 제거 검증 (난이도 ★★☆☆☆)

실습 목표

4장이 실제 Kafka 브로커에서 남긴 중복 스트림 40건(고유 30건, 재수신 중복 10건)을 네 단계로 적재해 저장 전략의 차이를 눈으로 확인한다. A는 제약 없는 삽입, B는 유니크 제약과 무시, C는 UPSERT, D는 C 상태에서 전체 스트림을 한 번 더 리플레이한다. 마지막 D단계가 "먹통 쓰기가 exactly-once 반영을 만든다"를 실측으로 증명한다.

입력 파일은 시뮬레이터가 만든 것이 아니라 4장 실습 4.1 시나리오 C의 실제 처리 기록 스냅샷이다. 스냅샷이 동봉되어 있어 브로커 없이 단독으로 실행된다. 표준 라이브러리 `sqlite3` 만 쓰므로 추가 설치도 없다.

실행 명령

```
cd practice/chapter5
python3 run_chapter5.py
```

핵심 코드 — B단계(유니크 제약 + 무시)

```
cur = conn.execute("INSERT OR IGNORE INTO complaints_unique VALUES (?, ?, ?, ?)", row)
ignored += 1 - cur.rowcount    # rowcount 0 = 유니크 충돌로 무시됨
```

전체 코드는 `practice/chapter5/code/5-1-deduplicate-events.py` 참고

실행 결과와 대조

표 5.1 저장 전략별 검증 결과(입력 40건, 고유 30건)

단계	행 수	관찰 지표	해석
A. naive INSERT	40	중복 행 10	상류 중복이 그대로 데이터가 된다
B. 유니크 + OR IGNORE	30	무시된 중복 10	중복은 사라졌지만 발생 사실도 사라졌다
C. UPSERT	30	총 관찰 40, 재관찰 이벤트 10, 지문 <code>eed1522a7ca844bd</code>	중복을 제거하면서 발생 기록은 남긴다
D. 전체 리플레이	30(불변)	총 관찰 80, 지문 <code>eed1522a7ca844bd</code> (C와 동일)	업무 상태가 재처리에 불변하다

출처는 `practice/chapter5/data/output/ch5_dedup_report.json` 이다.

A와 B의 행 수 차이 10이 3절의 명제를 확인한다. 저장소에 방어선이 없으면 4장의 재수신 중복 10건이 그대로 집계 대상이 된다. B와 C는 행 수가 30으로 같지만 남기는 정보가 다르다. B는 중복이 있었다는 사실까지 지우고, C는 `seen_count` 로 보존한다. C의 재관찰 이벤트 10이 4장 크래시 실험의 재수신 10건과 정확히 일치하므로, 저장소 지표만으로 상류 장애를 역추적할 수 있다.

D가 이 실습의 결론이다. 전체 스트림을 한 번 더 흘러 넣어도 행 수는 30 그대로이고, 업무 필드(event_id·파티션·오프셋·최초 관찰 시각)만으로 만든 지문 해시가 C와 D에서 `eed1522a7ca844bd` 로 같다. at-least-once 전달 위에서 멍등 쓰기가 만들어 내는 "정확히 한 번 반영"이란 바로 이것, 즉 업무 상태의 리플레이 불변성이다.

다만 한 가지를 정확히 구분해야 한다. 전달 관찰 메타데이터인 `seen_count` 와 `last_seen_at` 은 리플레이에서 총 관찰 40에서 80으로 **의도한 대로 늘어난다**. 전달 횟수의 기록이기 때문이다. UPSERT 테이블에서 alo-000003 한 행을 조회하면 first_seen_at 01:59:20, last_seen_at 01:59:35, seen_count 4가 나온다. 입력 스트림에서 두 번, D단계 리플레이에서 또 두 번 도착했지만 행은 하나다. 전체로 계산하면 중복이 없던 20건이 각각 2회, 중복 10건이 각각 4회로 $20 \times 2 + 10 \times 4 = 80$ 이 되어 표의 총 관찰 수와 맞는다.

7. 핵심 요약

1. 중복은 유실을 막으려는 재시도의 부작용이다. 확인 응답이 유실되면 Producer는 기록이 안 된 것인지 응답만 사라진 것인지 구분할 수 없고, 재전송을 포기하면 유실을 감수해야 한다. 4장이 남긴 중복 10건은 커밋 전 크래시로 컨슈머가 같은 (파티션, 오프셋)을 두 번 읽은 결과다.

2. exactly-once는 전달 보장이 아니라 반영 보장이다. 네트워크 위에서 "정확히 한 번 전달"은 원리적으로 보장하기 어렵고, 실무의 exactly-once는 여러 번 전달돼도 한 번 반영된 것과 같은 결과를 뜻한다. 그 결과를 만드는 것이 멍등 쓰기다.

3. 중복 제거 키는 생성 시점에 발급되어 재시도 경로에서 불변이어야 한다. 전송할 때마다 새 ID를 붙이면 "모든 이벤트에 고유 ID가 있다"는 요구는 만족하면서 중복 제거는 전혀 못 한다. 그리고 이 키가 지우는 것은 같은 이벤트의 반복 전달일 뿐, 시민의 재접수는 통과시키는 것이 정상이다.

4. 방어선은 코드가 아니라 스키마에 둔다. "확인 후 저장" 코드는 컨슈머 두 대가 동시에 움직이면 경쟁 조건에 뚫리지만, 유니크 제약은 데이터가 놓이는 단 한 곳에서 원자적으로 지켜진다. 충돌을 병합으로 처리하면 중복이 `seen_count` 라는 관찰 데이터로 바뀐다.

5. 멍든 반영은 지문 해시로 증명한다. 전체 스트림을 다시 흘러 넣어도 행 수는 30 그대로이고 업무 상태 지문은 `eed1522a7ca844bd` 로 같았다. 반면 `seen_count` 는 40에서 80으로 늘어나는데, 이것이 정상인 이유는 그것이 업무 상태가 아니라 전달 이력이기 때문이다.

8. 과제

실습 5.1을 실행해 만든 `practice/chapter5/data/output/ch5_dedup_report.json` 을 LMS에 업로드하고, 본인 산출물의 값을 인용해 다음 세 가지를 각각 세 문장 이내로 쓴다.

1. A단계와 C단계의 행 수를 옮겨 적고, 그 차이가 4장의 어느 실측과 대응하는지 쓴다.
2. C와 D의 지문 해시가 같은데 총 관찰 수는 40에서 80으로 늘어났다. 두 값이 서로 모순되지 않는 이유를 "업무 상태"와 "전달 이력"의 구분으로 설명한다.
3. "본 시스템은 Kafka를 사용하므로 메시지가 정확히 한 번 처리되며 유실과 중복이 발생하지 않는다"는 발주 문장을 본인 수치로 반박하고, 실행 증거로 확인할 수 있는 문장으로 고쳐 쓴다.

값을 지어내지 않는다. 실행이 막히면 어디서 막혔는지 그대로 쓴다. 수업일 포함 7일 이내에 제출한다.

9. 더 알아보기

이 장에서 자른 내용은 실무교재 `docs/chapter5.md` 에 있다.

- **지연 이벤트와 watermark** — 언제까지 기다릴지의 선언과 잠정치·확정치 구분: `docs/chapter5.md` §5.4 (6장에서 직접 구현한다)
- **키 설계 점검 체크리스트** — 키를 확정하기 전에 순서대로 확인할 다섯 항목: §5.2
- **재난 문자와 발송 아웃박스** — 되돌릴 수 없는 발송을 유니크 제약과 상태 컬럼으로 다루는 방법: §5.3
- **Kafka 트랜잭션과 Redis dedup 캐시의 TTL** — 보장의 경계가 어디까지인지, 얼마나 오래 기억해야 하는지: `docs/chapter5.md` 보충
- **ch5_dedup.sqlite 직접 질의** — 세 전략의 테이블이 한 DB에 공존하므로 `seen_count` 분포를 직접 확인할 수 있다: `practice/chapter5/`